



Fine-tuning LLMs — Từ Full Fine-tune đến LoRA/QLoRA

AICB-P2T3 · Ngày 21 · Chương 5 — Fine-tuning & An Toàn

Nguyễn Khánh Linh

VinUniversity · Phase 2 · Track 3 · Tuần 5

HÃY SUY NGHĨ...

“Khi nào nên fine-tune — và khi nào prompt engineering đủ rồi?”

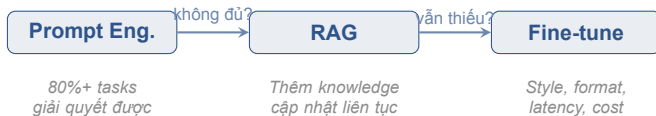
Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Khi nào cần Fine-tune?
2. LoRA — Cơ chế hoạt động
3. QLoRA — Fine-tune trên GPU nhỏ
4. Dataset & Training Pipeline
5. Demo & Thực hành

Khi nào cần Fine-tune?

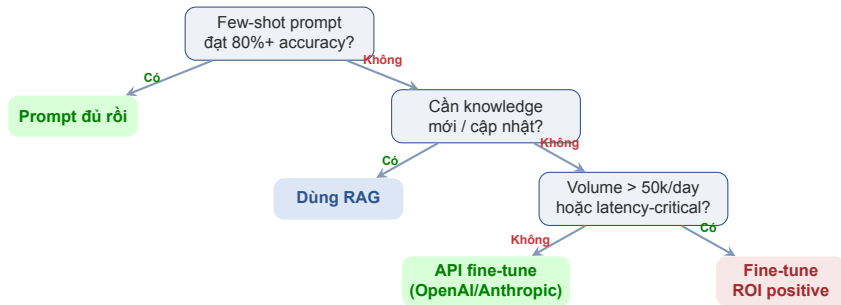
Prompt Engineering đã đủ chưa — hay cần huấn luyện thêm?

01



- Frontier models (GPT-4o, Claude 3.5, Gemini 2) đủ tốt cho **hầu hết tasks**
- Fine-tune chỉ khi thực sự cần: format riêng, domain jargon, giảm cost at scale
- Model options: **Qwen2.5-7B** hoặc **Gemma-2-9B**

Lưu ý: Fine-tune **KHÔNG** fix knowledge gaps — dùng RAG cho knowledge. Fine-tune fix *style* và *format*.



Prototype bằng prompting, đo gap. Nếu **gap** > **15%** và **volume** > **50k req/day** $\Rightarrow$ fine-tuning có ROI dương. Luôn thử prompt $\rightarrow$ RAG trước khi tới fine-tune.

Tiêu chí	API FT (OpenAI/Anthropic)	Self-host LoRA/QLoRA
Infra management	Không cần — managed	Tự setup GPU + pipeline
Control	Hạn chế (hyperparam)	Toàn quyền (rank, layers, data)
Cost per token	Cao (\$\$\$)	Thấp khi volume lớn
Latency	Mạng + queue	Self-host vLLM, sub-200ms
Data privacy	Gửi qua API provider	On-prem hoàn toàn
Time to production	Vài giờ	Vài ngày

Prototype nhanh, volume nhỏ–vừa, không có team ML ops, dữ liệu non-sensitive.

Volume > 50k/day, dữ liệu nhạy cảm, cần custom rank/layers, cần multi-tenant adapters.

LoRA — Cơ chế hoạt động

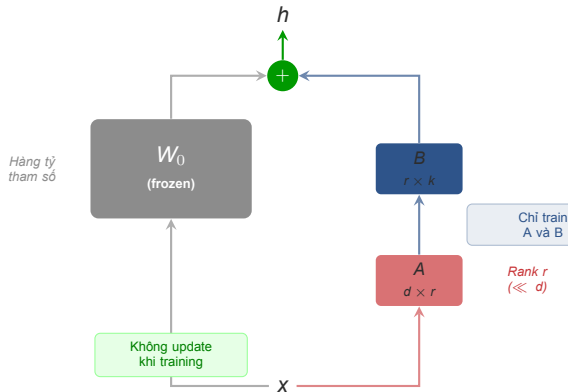
Low-Rank Adaptation: thêm ghi chú vào sách, không sửa sách gốc

LoRA (Hu et al. 2021)

Freeze toàn bộ base weights (hàng tỷ tham số), chỉ inject thêm **low-rank update** $\Delta W = B \cdot A$ — train phần rất nhỏ tham số.

Analogy: LoRA giống **thêm sticky notes vào sách giáo khoa** — không sửa sách gốc, chỉ thêm ghi chú nhỏ. Khi deploy, “dán” ghi chú vào sách $\Rightarrow$ zero added latency.

Inference: merge adapter vào base weights: $W = W_0 + B \cdot A$ — **không tốn thêm thời gian** khi chạy.



Toán học:

$$h = W_0 x + \underbrace{B \cdot A}_{\Delta W} \cdot x$$

Rank $r \ll \min(d, k)$. Thường $r \in \{8, 16, 32, 64\}$.

$\text{lora_alpha}/r = 1$ hoặc 2.
Alpha cao $\Rightarrow$ adapter ảnh hưởng mạnh hơn.

Target layers (2025 best practice):
ALL attention + MLP layers

q_proj + v_proj alone thiếu capacity

$r = 8$

0.1% params
Nhẹ, tiết kiệm

$r = 16$

Standard
Cân bằng tốt

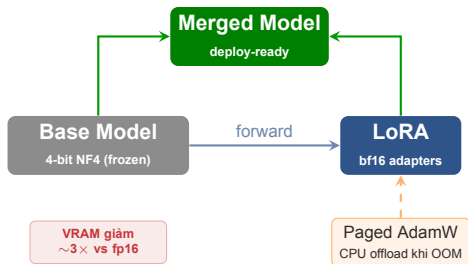
$r = 64$

Near full FT
Tốn VRAM hơn

Bắt đầu $r = 16$. Nếu accuracy chưa đủ $\Rightarrow$ tăng lên 32, 64. Nếu VRAM hạn chế $\Rightarrow$ giảm xuống 8. Variants mới: **DoRA**, **rsLoRA** — cải thiện nhẹ, cùng nguyên lý.

QLoRA — Fine-tune trên GPU nhỏ

4-bit quantization + LoRA = huấn luyện 7B trên RTX 3090



QLoRA (Dettmers et al. 2023) — Quantize base model xuống 4-bit NF4, thêm bf16 LoRA adapters. Chỉ train adapters.

- **Paged AdamW**: offload optimizer states sang CPU RAM khi GPU hết bộ nhớ
- **Double quantization**: quantize cả quantization constants — tiết kiệm thêm
- **Quality drop**: chỉ ~2–5% vs LoRA 16-bit

Phương pháp	VRAM (7B)	Params train	Thời gian	GPU tối thiểu
Full Fine-tune	~60 GB	100%	\$\$\$\$	A100 80GB
LoRA (fp16)	~28 GB	~1%	\$\$	A100 40GB
QLoRA (4-bit)	~10 GB	~1%	\$\$	RTX 3090 24GB

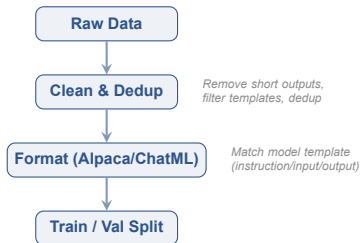
Train QLoRA → merge adapter →
quantize GGUF/AWQ → deploy vLLM
/ llama.cpp

1 base model + nhiều LoRA adapters
→ serve nhiều domains cùng lúc trên
1 GPU

Dataset & Training Pipeline

Từ chuẩn bị dữ liệu đến chạy training với Unsloth + TRL

04



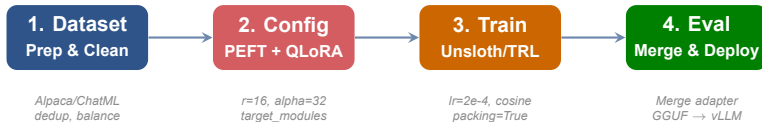
Quy mô cần thiết:

- Style/format: **500–2k** samples chất lượng cao
- Domain adaptation: **10k+** samples
- Rule: *500 perfect > 10k noisy*

Lưu ý: Data contamination: verify test set **KHÔNG** overlap training data. Đây là “silent killer” của eval reliability.

Synthetic data: GPT-4 Evol-Instruct cho 10× expansion. Cẩn thận *model collapse* nếu lạm dụng.

Distillation: generate training data từ strong model (GPT-4, Claude) → fine-tune weaker model. Hiệu quả — nhưng **kiểm tra ToS** của provider trước.



Model + Optimizer + Activations + Gradients.

QLoRA giảm ~60% vs full fp16.

Flash Attention 2: bắt buộc — 2–4× speedup.

RTX 3090 (24GB): 7B QLoRA

A100 (40GB): 13B QLoRA

H100 (80GB): 70B QLoRA

Grad. checkpoint: –60% VRAM, +20% time

VRAM budget breakdown:

$$V_{\text{tot}} = V_{\text{model}} + V_{\text{optim}} + V_{\text{act}} + V_{\text{grad}}$$

- **Flash Attention 2:** bắt buộc — 2–4× speedup, giảm activations memory đáng kể
- **Gradient checkpointing:** recompute activations khi backward → –**60% VRAM**, +20% time
- QLoRA + Flash Attn 2 + grad checkpoint ⇒ ~**10 GB** cho 7B model

Phân tích token distribution của dataset → set `max_seq_length` = p95. Tránh padding lãng phí.

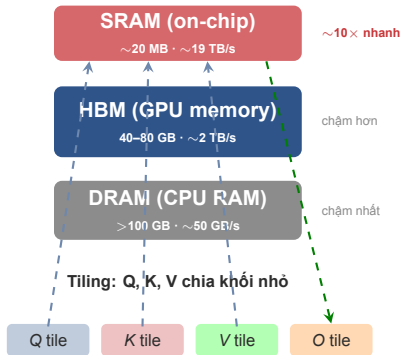
$\text{batch}_{\text{eff}} = \text{batch} \times \text{grad_accum}$
Khi VRAM hạn chế: `batch=1`, `grad_accum=4--8`
Mục tiêu: $\text{batch}_{\text{eff}} \in [16, 64]$

Lưu ý: OOM debugging: giảm `max_seq_length` trước, sau đó bật grad checkpointing,

Dao et al. 2022, FA2 (2023), FA3 (2024)

Vanilla attention đọc/ghi ma trận $N \times N$ qua HBM (slow) $\Rightarrow$ memory-bound. FlashAttention dùng **tiling + recomputation**: tải block nhỏ vào SRAM (on-chip, ~ 20 MB nhưng $\sim 10\times$ nhanh hơn HBM), tính softmax theo từng block, tránh materialize ma trận attention $N \times N$.

Kết quả: **exact attention** (không xấp xỉ) — nhưng **2–4 $\times$ nhanh hơn** và memory **$O(N)$** thay vì $O(N^2)$.



Vì sao nhanh hơn?

- Vanilla: ghi ma trận $N \times N$ vào HBM → memory-bound
- FA: streaming tiles qua SRAM → compute-bound
- Backward: *recompute* attention thay vì store → tiết kiệm activation memory

FA1 (2022): tiling cơ bản

FA2 (2023): better parallelism, +2× speed

FA3 (2024): Hopper TMA, FP8, cho H100

Hardware requirements:

- **Ampere+**: A100, RTX 30xx, RTX 40xx (FA2 OK)
- **Hopper**: H100, H200 (FA3 mới nhất, FP8)
- **Không hỗ trợ**: V100, T4 (chỉ Turing)

Sequence length sweet spot:

- Short (<512): speedup nhỏ, vẫn nên bật
- Medium (1k–4k): **2–3×** speedup
- Long (8k+): **4×+** speedup, memory tiết kiệm rất lớn

Cách bật trong code:

```
from_pretrained(  
    ...,  
    attn_implementation=  
    "flash_attention_2"  
)
```

Tự động bật FA2 ngay khi load model 4-bit. Không cần config.

Install: `pip install flash-attn --no-build-isolation`
(cần CUDA toolkit + nvcc, build ~10 phút)

```
from unsloth import FastLanguageModel
from trl import SFTTrainer
model, tok = FastLanguageModel.from_pretrained(
    "unsloth/Qwen2.5-7B-bnb-4bit",
    max_seq_length=2048, load_in_4bit=True)
model = FastLanguageModel.get_peft_model(
    model, r=16, lora_alpha=32,
    target_modules=["q_proj", "k_proj",
        "v_proj", "o_proj", "gate_proj",
        "up_proj", "down_proj"])
trainer = SFTTrainer(
    model=model, train_dataset=dataset,
    dataset_text_field="text",
    packing=True) # 2x throughput
trainer.train()
```

Key settings:

- **Unsloth:** custom CUDA kernels — $2\times$ faster, 60% less VRAM
- **packing:** gộp samples vào 1 sequence — $2\times$ throughput
- LR: $2e-4 \rightarrow 5e-5$, cosine schedule
- Warmup: 5–10% total steps

Lưu ý: Overfitting: eval loss tăng + train loss giảm $\Rightarrow$ **dừng training**, giảm epochs hoặc tăng data.

Demo & Thực hành

Xem fine-tuning hoạt động thực tế trên domain Việt Nam

05

1. Dataset: 1k Vietnamese QA pairs (Alpaca format). Training: 3 epochs, ~25 min on A100
2. Before: base model trả lời generic. After: dùng domain terminology chính xác
3. LoRA adapter swap: load khác adapters cho khác domains — multi-tenant serving
4. GGUF demo: merge adapter → convert → run llama.cpp — deployment ready

Mục tiêu: Fine-tune Qwen2.5-7B với LoRA/QLoRA trên custom Vietnamese dataset (dùng Unsloth + TRL)

Deliverable: LoRA adapter checkpoint + evaluation report: perplexity delta, 5 qualitative before/after examples, training cost — so sánh rank $r = 8$ vs $r = 64$

Thời gian: 2 giờ

1. **Data prep** (15p): chuẩn bị 100–500 examples Alpaca format từ domain. Clean, tokenize, verify `max_seq_length` (p95), split 90/10 train/eval
2. **Configure PEFT** (10p): `r=16`, `alpha=32`, target `q_proj+v_proj`. Setup QLoRA 4-bit với `Unsloth FastLanguageModel`
3. **Train baseline** (40p): TRL `SFTTrainer`, 3 epochs, `packing=True`, cosine LR, warmup 10%. Monitor loss curve — detect overfitting (eval loss rising)
4. **Rank experiment** (30p): train 2 thêm adapters với `r=8` và `r=64` trên cùng dataset. So sánh: training time, VRAM usage, eval perplexity, qualitative output
5. **Evaluate** (15p): perplexity trên eval set, generate 20 test prompts, qualitatively compare fine-tuned vs base model

GitHub repo + evaluation report:

(1) LoRA adapter checkpoints (3 ranks) (2) Bảng perplexity delta + training cost (3) 5 qualitative before/after examples (4) Kết luận về rank selection trade-off

Cấu hình	Train time	Peak VRAM	Eval PPL	Qualitative
Base (no FT)	—	—	?	generic
LoRA $r = 8$	?	?	?	?
LoRA $r = 16$	?	?	?	?
LoRA $r = 64$	?	?	?	?

Rank nào cho ROI tốt nhất trên dataset của bạn? Khi nào tăng rank không còn cải thiện perplexity (*diminishing returns*)? Khi nào nên chọn $r = 8$ thay vì $r = 16$?

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 QLoRA democratize fine-tuning — consumer GPU (RTX 3090) đủ cho 7B–8B models
- 2 Dataset quality là yếu tố quan trọng nhất — 500 perfect > 10k noisy
- 3 Sequence packing là free $2\times$ speedup — luôn bật `packing=True`
- 4 LoRA adapters are composable — serve nhiều adapters cùng lúc trên 1 base model

Ngày 22: DPO, ORPO & Alignment

“SFT dạy format — DPO/ORPO dạy alignment (helpful + safe). RLHF tốn kém — DPO có thể thay thế?”

- Hoàn thành Lab 21: LoRA fine-tune + benchmark report
- Đọc: Rafailov et al. “Direct Preference Optimization” (2023)

Hỏi & Đáp

Khi nào thì fine-tune thực sự cần thiết? Bạn có use case cụ thể nào muốn thảo luận?



Cảm ơn!

AICB-P2T3 · Ngày 21 · Fine-tuning LLMs
— Từ Full Fine-tune đến LoRA/QLoRA

github.com/vinuni-aicb

Liên hệ: instructor@vinuni.edu.vn